Computational Astrophysics Exercises

Assignments Set 3 — Summer Term 2026

Names: Karl Gauck, 6754404
Benedict Sondershaus, 6678384

Date: 15.05.26

Exercise 1: Catastrophic cancellation — 3 pts

(a) The naive solution is implemented in the method `classic_solution(a, b, c)`. It returns both roots given by

$$r_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{1}$$

as an optional value as tuple. If the square root is not real, an empty option will be returned instead.

We are only interested in the first value with $-b + \sqrt{\dots}$.

(b) The following plot shows the errors for finding the roots with parameters $a = 1, b = 1, c = 10^{-n}, n = 1..25$. It shows the errors for single (f32) and double (f64) precision, together with the respective machine epsilon. The error is calculated against the "stable" method, calculated with a float with 512 bits (16x precision) to make sure the values are accurate.

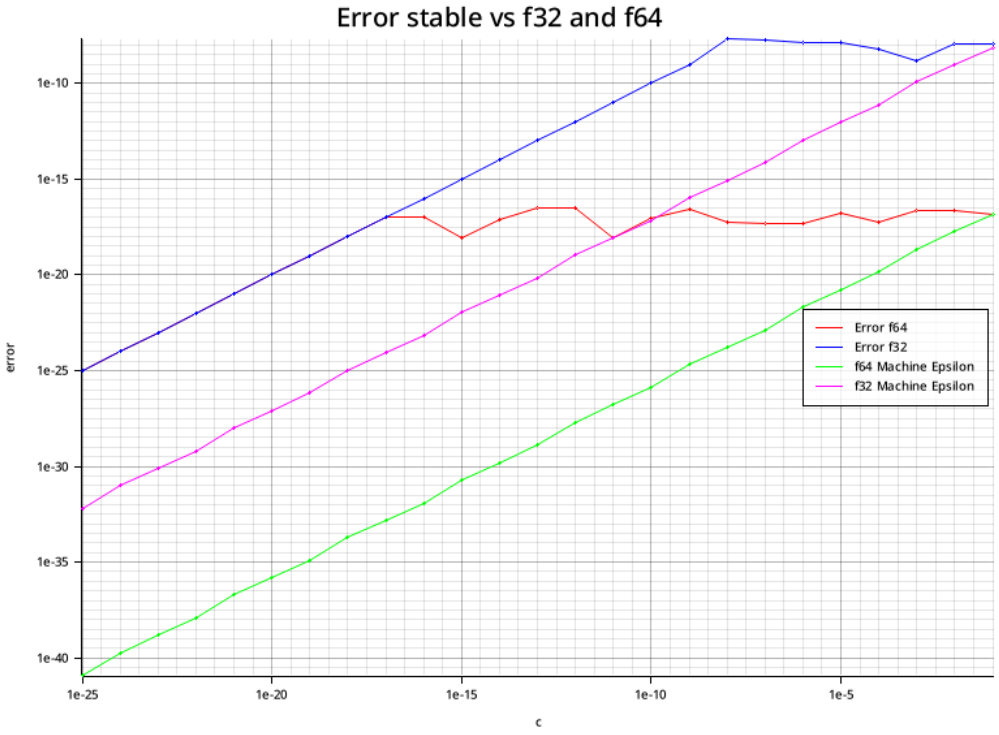

Figure 1: Error plots of the roots for $a = 1, b = 1, c = 10^{-n}, n = 1..25$, c is on the x-axis, the error is on the y-axis. Both axes are in log scale

We could not observe a real "explosive" increase of error at a certain point, only a plateau, where the error lines diverge from the machine epsilon to another greater value, let's call it $e'(c)$. This $e(c)$ is the same for f32 and f64 and again decreases for decreasing c. The machine epsilon values or values in its surrounding would be expected as absolute error, but the difference between the machine epsilon and $e(c)$ is of a magnitude of ~ 16.

So this plateau is probably caused by the catastrophic cancellation as in this range $b^2 \approx \sqrt{b^2 - 4ac}$ holds and the difference cannot be described by the floating point precision. For even smaller values of c (around 10^{-17} for double precision), the precision of the floating point values increases again, and thus the error decreases.

(c) The stable solution is implemented with **Vieta's formulas**. They state that for a polynomial of degree 2 in the representation $P(x) = ax^2 + bx + c$ the following holds about its roots r_1, r_2:

$$r_1 + r_2 = -\frac{b}{a} \tag{2}$$

$$r_1 r_2 = \frac{c}{a} \tag{3}$$

Our problem is if $b \gg 4ac$ as the first root is calculated via:

$$r_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \tag{4}$$

and thus b and the square root will cancel without the required precision. But the second root can be calculated without problems:

$$r_2 = -\frac{b + \sqrt{b^2 - 4ac}}{2a} \tag{5}$$

We can use this together with Equation 3 to calculate r_1:

$$\begin{aligned} r_1 &= \frac{c}{r_2 a} \\ &= \frac{-2c}{b + \sqrt{b^2 - 4ac}} \end{aligned} \tag{6}$$

Thus the potential for cancellation is eliminated.

This stable solution is implemented in `other_stable_solution(a,b,c)` for 512-bit floats.

(d) The stable solution from **(c)** is again susceptible to catastrophic cancellation when $|b| \gg 4ac$ and $b < 0$, as then $b + \sqrt{b^2 - 4ac} \approx 0$. Thus we simply check if b is negative and if it is, we return the naive solution for r_1 and a similar solution like the stable one with Vieta for r_2:

$$r_2 = \frac{2c}{-b + \sqrt{b^2 - 4ac}} \tag{7}$$

The implementation can be found in `final_stable_solution(a,b,c)`

Exercise 2: Interpolation 7 pts

(a) The Runge function $f(x) = \frac{1}{1+x^2}$ is interpolated on $[-5, 5]$ using Newton interpolation for polynomial degrees $n = 12$ and $n = 20$. The divided difference coefficients are computed in the method `make_pol_line(n)` via the given pseudocode.

With equidistant nodes the **Runge phenomenon** is visible: Both polynomials diverge near the endpoints $x = \pm 5$, with $n = 20$ being surprisingly worse than $n = 12$ (so higher degree doesn't always increase precision).

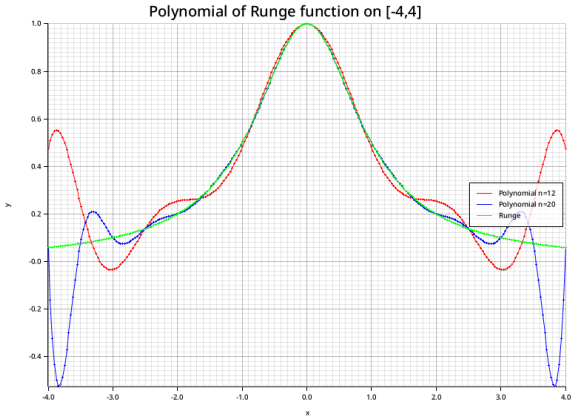

Figure 2: Interpolation polynomials P_{12} and P_{20} with equidistant nodes on $[-4, 4]$ for better visibility of the interpolation. Runge phenomenon is already visible at the edges.

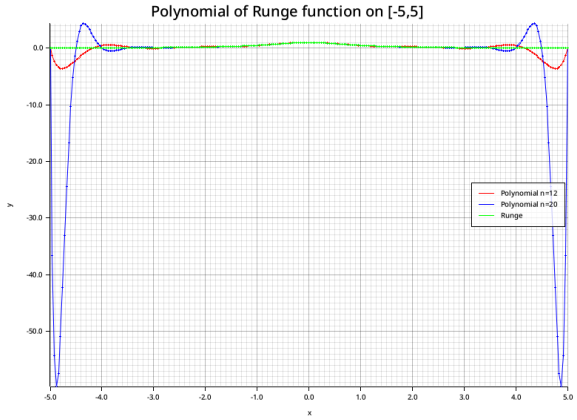

Figure 3: Interpolation polynomials P_{12} and P_{20} with equidistant nodes on $[-5, 5]$. Boundary divergence is strongly amplified compared to $[-4, 4]$.

(b) $W(x) = \prod_{i=0}^{n}(x - x_i)$ is computed in `make_W_line(n)` and plotted alongside all other plots. With equidistant nodes it grows to values on the order of 10^{12} near the interval edges for $n = 12$, and the $n = 20$ W-function oscillates with even larger amplitude across the whole interval.

(c) With Chebyshev nodes the $W(x)$ function is orders of magnitude smaller and the error is spread across the interval rather than spiking at the edges.

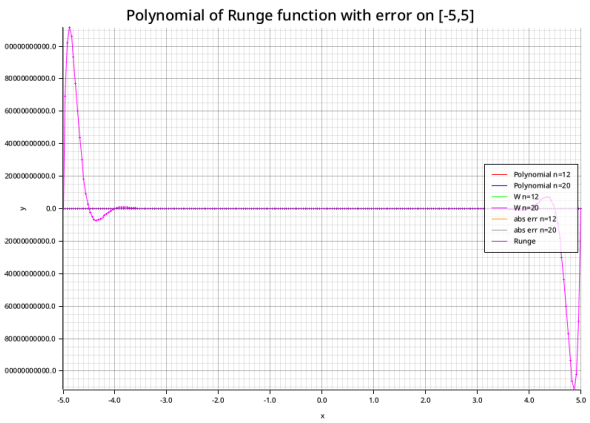

Figure 4: Interpolation error and $W(x)$ with equidistant nodes. The scale reaches $\sim 10^{12}$, dominated by boundary blow-up.

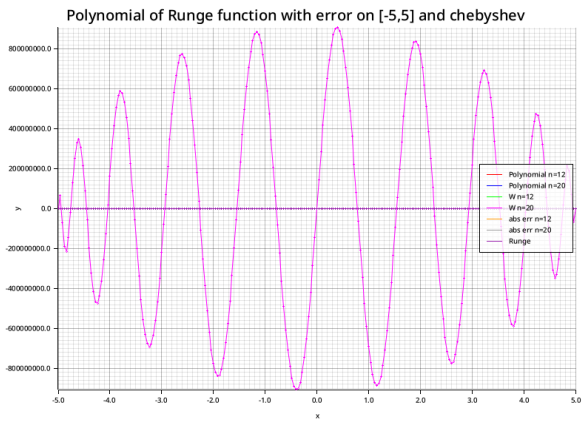

Figure 5: Interpolation error and $W(x)$ with Chebyshev nodes. The boundary blow-up is not visible anymore and $W(x)$ is spread along the interval.

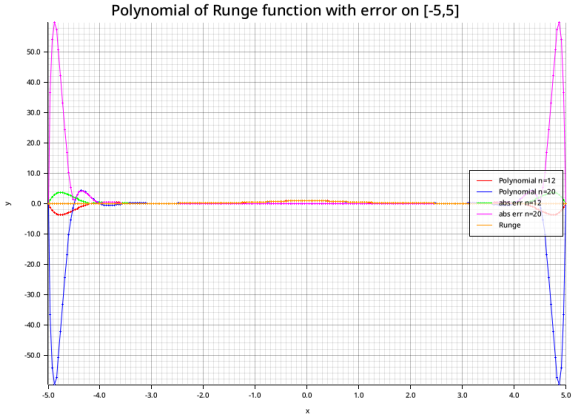

Figure 6: Absolute interpolation error with equidistant nodes. The error is largest near $x = \pm 5$ and increases with degree.

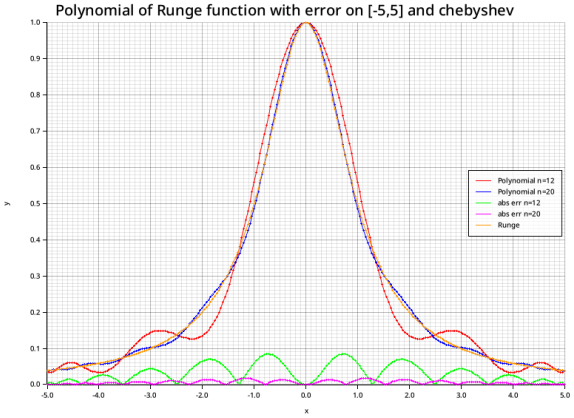

Figure 7: Absolute interpolation error with Chebyshev nodes. The error is small and evenly distributed across the interval for both degrees.